ECE 7670
Program 5
Nathan Walker

November 3, 2025

---

1. Preliminary Exercises 1, 2, 3 are asked to work through and verify the berlekamp-Massey algorithm with given sequences

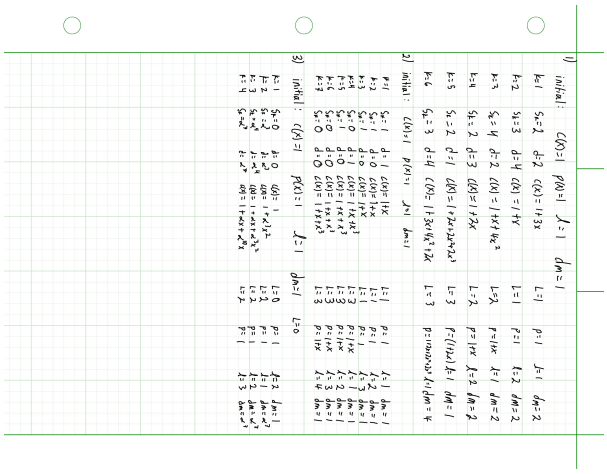


Figure 1: Hand work of the preliminary exercises

2. write a function berlmass which can accept a sequence of numbers and return either a polynomial or an array of connection polynomials coefficients.

3. use your function to verify the preliminary results

4. for Exercises 3, 4, 5 use your function to verify the connection polynomials given the following sequences of data. for exercise 5 use the BCH method to reduce complexity.

```
$ ./testBM.exe
k=0 s=1 d=1  c=1 + x  L=1 p=1  l=1 dm=1
k=1 s=1 d=0  c=1 + x  L=1 p=1  l=2 dm=1
k=2 s=1 d=0  c=1 + x  L=1 p=1  l=3 dm=1
k=3 s=0 d=1  c=1 + x + x^3  L=3 p=1 + x  l=1 dm=1
k=4 s=1 d=0  c=1 + x + x^3  L=3 p=1 + x  l=2 dm=1
k=5 s=0 d=0  c=1 + x + x^3  L=3 p=1 + x  l=3 dm=1
k=6 s=0 d=0  c=1 + x + x^3  L=3 p=1 + x  l=4 dm=1
prelim (2) GF2 polynomial form: c=1 + x + x^3

k=0 s=2 d=2  c=1 + 3x  L=1 p=1  l=1 dm=2
k=1 s=3 d=4  c=1 + x  L=1 p=1  l=2 dm=2
k=2 s=4 d=2  c=1 + x + 4x^2  L=2 p=1 + x  l=1 dm=2
k=3 s=2 d=3  c=1 + 2x  L=2 p=1 + x  l=2 dm=2
k=4 s=2 d=1  c=1 + 2x + 2x^2 + 2x^3  L=3 p=1 + 2x  l=1 dm=1
k=5 s=3 d=4  c=1 + 3x + 4x^2 + 2x^3  L=3 p=1 + 2x  l=2 dm=1
prelim (1) Z5 c=1 + 3x + 4x^2 + 2x^3

k=0 s=0 d=0  c=1  L=0 p=1  l=2 dm=1
k=1 s=A^3 d=A^3  c=1 + A^3x^2  L=2 p=1  l=1 dm=A^3
k=2 s=A^4 d=A^4  c=1 + Ax + A^3x^2  L=2 p=1  l=2 dm=A^3
k=3 s=A^7 d=1  c=1 + Ax + A^10x^2  L=2 p=1  l=3 dm=A^3
prelim (3) GF(16) c=1 + Ax + A^10x^2

k=0 s=A^8 d=A^8  c=1 + A^8x  L=1 p=1  l=1 dm=A^8
k=1 s=A d=0  c=1 + A^8x  L=1 p=1  l=2 dm=A^8
k=2 s=A^13 d=A^10  c=1 + A^8x + A^2x^2  L=2 p=1 + A^8x  l=1 dm=A^10
k=3 s=A^2 d=0  c=1 + A^8x + A^2x^2  L=2 p=1 + A^8x  l=2 dm=A^10
k=4 s=A^5 d=0  c=1 + A^8x + A^2x^2  L=2 p=1 + A^8x  l=3 dm=A^10
k=5 s=A^11 d=0  c=1 + A^8x + A^2x^2  L=2 p=1 + A^8x  l=4 dm=A^10
problem (3) GF(16) c=1 + A^8x + A^2x^2

k=0 s=0 d=0  c=1  L=0 p=1  l=2 dm=1
k=1 s=0 d=0  c=1  L=0 p=1  l=3 dm=1
k=2 s=A^5 d=A^5  c=1 + A^5x^3  L=3 p=1  l=1 dm=A^5
k=3 s=0 d=0  c=1 + A^5x^3  L=3 p=1  l=2 dm=A^5
k=4 s=1 d=1  c=1 + A^10x^2 + A^5x^3  L=3 p=1  l=3 dm=A^5
k=5 s=A^10 d=0  c=1 + A^10x^2 + A^5x^3  L=3 p=1  l=4 dm=A^5
problem (4) GF(16) c=1 + A^10x^2 + A^5x^3

k=0 s=1 d=1  c=1 + x  L=1 p=1  l=2 dm=1
k=2 s=A^10 d=A^5  c=1 + x + A^5x^2  L=2 p=1 + x  l=2 dm=A^5
k=4 s=A^10 d=A^10  c=1 + x + A^5x^3  L=3 p=1 + x + A^5x^2  l=2 dm=A^10
problem 5(a) GF(16) c=1 + x + A^5x^3

k=0 s=A^14 d=A^14  c=1 + A^14x  L=1 p=1  l=2 dm=A^14
k=2 s=1 d=A^11  c=1 + A^14x + A^12x^2  L=2 p=1 + A^14x  l=2 dm=A^11
k=4 s=A^5 d=A^11  c=1 + A^14x + A^11x^2 + A^14x^3  L=3 p=1 + A^14x + A^12x^2  l=
2 dm=A^11
problem 5(b) GF(16) c=1 + A^14x + A^11x^2 + A^14x^3
```

Figure 2: output of terminal of all the exercises.

```
//  Program: berlmass.cc --- implement the Berlekamp-Massey algorithm
//  Todd K. Moon

#ifndef BERLMASS_CC
#define BERLMASS_CC

#include "polynomialT.h"

template <class T> polynomialT<T>
berlmassBCH(const T* s, int n)
{
    int L = 0;
   polynomialT<T> c;
   c[0] = T(1);
   // cout << "c=" << c << endl;
   polynomialT<T> p(c), t;      // p is previous, t is temporary
   int shift = 1;               // n-m
   T dm = T(1);                 // previous discrepancy
   T d;                         // current discrepancy
   int j;
   for(int k=0; k < n; k=k+2) {
        T sum = T(0);
        for(int i = 0; i < L; i++)
        {
            if( (i + 1) <= c.getdegree())
            {
                // cout << "c[i+1]=" << c[i + 1] << " s[k-i]=" << s[k-i] << endl;
                sum += c[i+1]*s[k-i-1];
            }
        }
        d = s[k] + sum;
        // cout << "s[k]=" << s[k] << " sum=" << sum << endl;
        if (d == 0)
            shift++;
        else
            if((2*L) >= (k+1))
            {
                polynomialT<T> xl(T(0));
                T d_prod = d/dm;
                // cout << "dm=" << dm << " d=" << d << " d_prod=" << d_prod << endl;
                xl.setc(shift, d_prod);
                c = c - (xl * p);
                shift++;
            }
            else
            {
                t = c;
```

```
                polynomialT<T> xl(T(0));
                T d_prod = d/dm;
                xl.setc(shift, d_prod);
                c = c - (xl * p);
                L = (k+1) - L;
                p = t;
                dm = d;
                shift = 1;
            }
        shift++;
      // fill in the blanks ....

cout<<"k="<<k << " s=" << s[k] << " d=" << d << "  c=" << c << "  L=
    " << L;
cout << " p=" << p << "  l=" << shift << " dm=" << dm << endl;

   }
    // cout << "exiting function" << endl;
   return c;
}

template <class T> polynomialT<T>
berlmass(const T* s, int n)
// s = input coefficients s[0],s[1],... s[n-1]
// returns = final connection polynomial
{
   int L = 0;
   polynomialT<T> c;
   c[0] = T(1);
   // cout << "c=" << c << endl;
   polynomialT<T> p(c), t;      // p is previous, t is temporary
   int shift = 1;               // n-m
   T dm = T(1);                 // previous discrepancy
   T d;                         // current discrepancy
   int j;
   for(int k=0; k < n; k++) {
        T sum = T(0);
        for(int i = 0; i < L; i++)
        {
            if( (i + 1) <= c.getdegree())
            {
                // cout << "c[i+1]=" << c[i + 1] << " s[k-i]=" << s[
                   k-i] << endl;
                sum += c[i+1]*s[k-i-1];
            }
        }
        d = s[k] + sum;
        // cout << "s[k]=" << s[k] << " sum=" << sum << endl;
        if (d == 0)
            shift++;
        else
```

```
97              if((2*L) >= (k+1))
98              {
99                   polynomialT<T> xl(T(0));
100                  T d_prod = d/dm;
101                  // cout << "dm=" << dm << " d=" << d << " d_prod="
                        << d_prod << endl;
102                  xl.setc(shift, d_prod);
103                  c = c - (xl * p);
104                  shift++;
105             }
106             else
107             {
108                  t = c;
109                  polynomialT<T> xl(T(0));
110                  T d_prod = d/dm;
111                  xl.setc(shift, d_prod);
112                  c = c - (xl * p);
113                  L = (k+1) - L;
114                  p = t;
115                  dm = d;
116                  shift = 1;
117             }
118       // fill in the blanks ....
119
120 cout<<"k="<<k << " s=" << s[k] << " d=" << d << "  c=" << c << "  L=
      " << L;
121 cout << " p=" << p << "  l=" << shift << " dm=" << dm << endl;
122
123    }
124     // cout << "exiting function" << endl;
125    return c;
126 }
127
128
129 #endif
130 /*
131 Local Variables:
132 compile-command: "g++ -g berlmass.cc"
133 End:
134 */
```

Listing 1: berlmass.cc